To filter the `Venue` listing, there are a couple of wrinkles. First, you need to use the `Optional` type hint to specify that this query parameter isn't required. Second, by default, the filter is an exact match. To perform starts-with, you need some extra stuff. Ninja uses Pydantic's `Field` class to give you more control over how a field behaves. Without the `Field` class, this would simply match the name of the field. With it, you can specify the `q` argument to a `QuerySet.filter()` call, using the `__istartswith` field look-up you learned about in chapter 4. The following listing contains the `VenueFilter` code.

Listing 11.9 Declare the fields that participate in a Ninja filter

```
# RiffMates/bands/api.py
...
from typing import Optional
...
from ninja import Field, Router, ModelSchema,
➥ FilterSchema

...

class VenueFilter(FilterSchema):
    name: Optional[str] = Field(None,
        q=['name__istartswith'])
```

Add Field and FilterSchema to the list of imports from ninja.

Inherit from FilterSchema to create a filter class.

Declare the name field, using Python type hinting to define it as an optional string and the Field class to change how it filters.

With the filter class ready, you need to make changes to the `Venue` listing endpoint. Replace the `name` argument of the function with the more generic `filters`. The key to Ninja understanding this is the Python type hint you provide. Ninja includes a `Query` type hint, which you combine with your `VenueFilter` class to specify that `filters` is a query filter.

Type hinting, a little deeper

The next chunk of code includes a bit of magic: Python's ellipsis (...). If you're not very familiar with type hinting, or don't really care, you can simply type it in as it is, and you'll be fine.

Type hinting is all about telling your tools what kinds of values to expect. For a complex filter, you could have a variable number of arguments. For `VendorFilter`, there is only name, but for a heavier object, you could filter on many fields.

The ellipsis literal in Python acts as a placeholder. You've seen it in the sample code, indicating I've skipped over stuff. The wonder of the ellipsis is that it is valid Python. If you copy and paste some sample code containing ..., Python won't complain. The ellipsis commonly gets used to create stub functions as well: define the function header, and make the body just ....

In type hinting, the ellipsis literal indicates a variable number of arguments. As the filter can have zero or more arguments, the syntax for the type hint is `filters: VendorFilter = Query(...)`.